

REGRAS LÓGICAS — NCAS

Núcleo Cognitivo da Aurora Siger (NCAS)

1. OBJETIVO

A camada lógica do NCAS realiza uma primeira triagem determinística dos alertas operacionais. Ela recebe os dados dos módulos da colônia, transforma condições operacionais em variáveis booleanas e aplica uma regra composta para identificar situações potencialmente críticas.

O resultado da lógica booleana não representa, sozinho, a decisão da Inteligência Artificial. Ele funciona como uma camada de validação e triagem, fornecendo informações estruturadas para a etapa posterior de interpretação simulada da IA.

2. VARIÁVEIS BOOLEANAS

Para realizar a análise, o sistema utiliza cinco variáveis:

F - Falha de sensor	=> F = NOT sensor_ok
E - Alta criticidade	=> E = criticidade >= 8
R - Area nao livre	=> R = NOT area_livre
C - Combustivel baixo	=> C = combustivel < 40
P - Alta prioridade	=> P = prioridade <= 2

F representa uma falha no sensor do módulo. E representa criticidade operacional igual ou superior a 8. R representa indisponibilidade da área necessária à operação. C representa nível de combustível abaixo de 40%. P representa prioridade operacional 1 ou 2.

3. REGRA BOOLEANA PRINCIPAL

A regra implementada no NCAS é:

```
FALHA_CRITICA = (F AND E) OR (R AND E) OR (C AND P)
```

A expressão determina que um alerta é crítico quando pelo menos uma das condições ocorrer:

- Existe falha no sensor E o módulo possui alta criticidade.
- A área de operação não está livre E o módulo possui alta criticidade.
- O combustível está abaixo de 40% E o módulo possui prioridade 1 ou 2.

4. SIMPLIFICAÇÃO — TEOREMA DE DE MORGAN

Além de saber quando um alerta é crítico, o NCAS também precisa saber quando uma situação é segura (ou seja, quando NÃO há falha crítica) — essa condição é usada pela camada de IA simulada para classificar alertas como ROTINA, sem exigir revisão humana obrigatória. Essa condição é obtida negando a regra principal e simplificando o resultado com o Teorema de De Morgan.

Partimos da negação da regra original:

```
NOT(FALHA_CRITICA) = NOT[ (F AND E) OR (R AND E) OR (C AND P) ]
```

Aplicando a 1a Lei de De Morgan -> NOT(A OR B OR C) = NOT(A) AND NOT(B) AND NOT(C):

```
NOT(FALHA_CRITICA) = NOT(F AND E) AND NOT(R AND E) AND NOT(C AND P)
```

Aplicando a 2a Lei de De Morgan a cada termo -> $\text{NOT}(X \text{ AND } Y) = \text{NOT}(X) \text{ OR } \text{NOT}(Y)$:

$\text{NOT}(\text{FALHA_CRITICA}) = (\text{NOT } F \text{ OR } \text{NOT } E) \text{ AND } (\text{NOT } R \text{ OR } \text{NOT } E) \text{ AND } (\text{NOT } C \text{ OR } \text{NOT } P)$

O que define a variável derivada utilizada pelo sistema:

$\text{SITUACAO_SEGURA} = (F' \text{ OR } E') \text{ AND } (R' \text{ OR } E') \text{ AND } (C' \text{ OR } P')$

4.1 Por que a simplificação mantém o mesmo resultado lógico

As Leis de De Morgan são identidades da álgebra booleana: valem para toda combinação possível de valores verdade, e não apenas para casos específicos. Isso pode ser conferido com uma tabela-verdade de 2 variáveis para cada lei:

A	B	$\text{NOT}(A \text{ OR } B)$	$\text{NOT}(A) \text{ AND } \text{NOT}(B)$
V	V	F	F
V	F	F	F
F	V	F	F
F	F	V	V

A	B	$\text{NOT}(A \text{ AND } B)$	$\text{NOT}(A) \text{ OR } \text{NOT}(B)$
V	V	F	F
V	F	V	V
F	V	V	V
F	F	V	V

Como as duas colunas de cada tabela são idênticas em todas as linhas, as leis são equivalências lógicas verdadeiras. Logo, aplicá-las sobre `FALHA_CRITICA` produz uma expressão (`SITUACAO_SEGURA`) logicamente equivalente a `NOT(FALHA_CRITICA)` para qualquer módulo da colônia — nunca apenas parecida ou coincidente.

5. VERIFICAÇÃO NOS MÓDULOS DA COLÔNIA

Para confirmar a equivalência na prática, comparamos `FALHA_CRITICA` com `NOT(SITUACAO_SEGURA)` em três módulos reais da colônia.

Exemplo 1 — HABIT-02

Dados: `sensor_ok=falso`, `criticidade=7`, `area_livre=verdadeiro`,
`combustivel=30%`, `prioridade=2`

Variáveis: `F=V` `E=F` `R=F` `C=V` `P=V`

$\text{FALHA_CRITICA} = (F \text{ AND } E) \text{ OR } (R \text{ AND } E) \text{ OR } (C \text{ AND } P)$
 $= (F) \text{ OR } (F) \text{ OR } (V) = \text{VERDADEIRO}$

$\text{SITUACAO_SEGURA} = (F' \text{ OR } E') \text{ AND } (R' \text{ OR } E') \text{ AND } (C' \text{ OR } P')$
 $= (F \text{ OR } V) \text{ AND } (V \text{ OR } V) \text{ AND } (F \text{ OR } F)$
 $= V \text{ AND } V \text{ AND } F = \text{FALSO}$

Verificacao: $\text{NOT}(\text{FALHA_CRITICA}) = \text{NOT}(V) = F = \text{SITUACAO_SEGURA}$ [OK]

Exemplo 2 — LAB-01

Dados: `sensor_ok=verdadeiro`, `criticidade=7`, `area_livre=falso`,
`combustivel=60%`, `prioridade=3`

Variáveis: `F=F` `E=F` `R=V` `C=F` `P=F`

```
FALHA_CRITICA = (F AND E) OR (R AND E) OR (C AND P)
               = (F) OR (F) OR (F) = FALSO
```

```
SITUACAO_SEGURA = (F' OR E') AND (R' OR E') AND (C' OR P')
                  = (V OR V) AND (F OR V) AND (V OR V)
                  = V AND V AND V = VERDADEIRO
```

Verificacao: NOT(FALHA_CRITICA) = NOT(F) = V = SITUACAO_SEGURA [OK]

Exemplo 3 — LOG-02

Dados: sensor_ok=verdadeiro, criticidade=5, area_livre=falso,
combustivel=20%, prioridade=3

Variaveis: F=F E=F R=V C=V P=F

```
FALHA_CRITICA = (F AND E) OR (R AND E) OR (C AND P)
               = (F) OR (F) OR (F) = FALSO
```

```
SITUACAO_SEGURA = (F' OR E') AND (R' OR E') AND (C' OR P')
                  = (V OR V) AND (F OR V) AND (F OR V)
                  = V AND V AND V = VERDADEIRO
```

Verificacao: NOT(FALHA_CRITICA) = NOT(F) = V = SITUACAO_SEGURA [OK]

Nos três módulos, SITUACAO_SEGURA coincide exatamente com a negação de FALHA_CRITICA, confirmando empiricamente a equivalência já demonstrada formalmente pelo Teorema de De Morgan.

6. SIMPLIFICAÇÃO COMPLEMENTAR — LEI DISTRIBUTIVA (FATORAÇÃO)

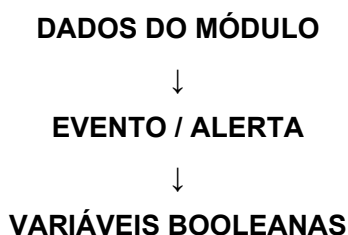
Além do Teorema de De Morgan, a regra principal também pode ser reorganizada com a Lei Distributiva da álgebra booleana, reduzindo o número de vezes que a variável E é escrita na expressão:

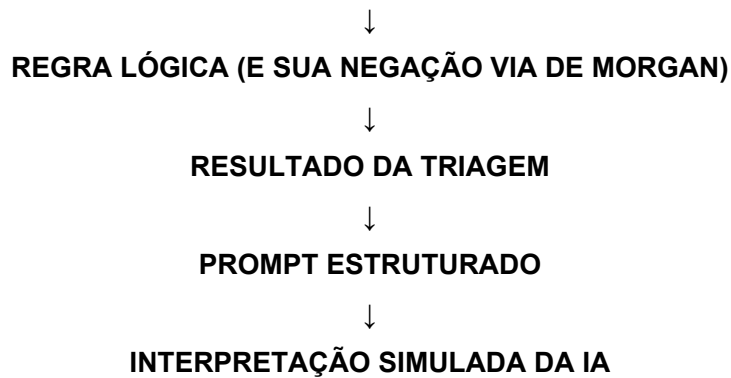
```
FALHA_CRITICA = (F AND E) OR (R AND E) OR (C AND P)
               = E AND (F OR R) OR (C AND P)
```

Esse reagrupamento não é uma aplicação do Teorema de Simplificação nem de De Morgan — é a Lei Distributiva, outro teorema fundamental da álgebra booleana. Ele é apresentado aqui apenas como demonstração complementar: o fator E passa a ser escrito uma única vez, evidenciando que ele é compartilhado pelas duas primeiras condições de risco (falha de sensor e área indisponível), enquanto o fator C permanece combinado apenas com P.

7. PAPEL DA LÓGICA NO NCAS

A lógica booleana representa a primeira camada de análise do sistema. O fluxo utilizado é:





Dessa forma, o sistema combina regras determinísticas com uma camada posterior de interpretação contextual. A lógica identifica condições objetivas previamente definidas, enquanto a camada de IA utiliza essas informações junto aos demais dados disponíveis para produzir uma resposta estruturada.

8. SUPERVISÃO HUMANA

Quando uma situação é classificada como crítica ou apresenta anomalias relevantes, o sistema indica:

REVISÃO HUMANA: SIM

Essa regra busca garantir que decisões operacionais críticas permaneçam sob supervisão humana. A automação tem como objetivo apoiar a tomada de decisão, e não substituir completamente o responsável pela operação.